

COMPUTING STUDENTS – PLACEMENT YEAR LOG REPORT 3

Student Name:	Mohammed Ali Mohammed Manazir
Placement Provider & Department:	Goodness Exchange
Workplace Supervisor Name:	Mr. Azim Abdul Majeed
Report Date:	April 21st, 2026
Reporting Period	February 1st, 2026 – April 20st, 2026

1. Description of Activities, Projects, Tasks, Roles, and Achievements

These past three months have been the densest stretch of my placement. The reporting period closes mid week on April 20th rather than at a clean week boundary, because the 19th and 20th carried a design marathon and a brand redesign pass that I wanted on the record.

Three streams of work ran in parallel almost every day. Protocol blockchain and full-stack engineering stayed the dominant share, as it has been since the start of the placement. Design and wireframing grew from a peripheral concern into a substantial track of its own once I began authoring participant facing surfaces directly. Marketing content development, one of the formal responsibility areas in my offer letter and an occasional contribution earlier in the placement, expanded into a daily operational rhythm alongside the founder. Holding all three streams together meant evenings, weekends, government holidays, and designated off days were routinely folded into the work. Three parallel tracks with hard dates do not leave room to clock off, and that extended cadence was structural rather than occasional.

Across the period I delivered a dual codebase consolidation, a blockchain migration off Hyperledger Fabric onto Hyperledger Besu with a Solidity Diamond implementation, a full enterprise design system rebuild, an end-to-end transfer pipeline, several administrative modules, an autonomous marketing daemon, and roughly two hundred and eighty wireframe screens authored in the final week across seven surfaces, the participant, institute, and government wallets, the institutional unified admin surface, the administrative command centre, a future modules roadmap, and a shared cross cutting primitives catalogue. The three-stream structure and the engineering discipline that held it together are the throughline of the narrative that follows.

February 2026: Consolidation, Hardening, and a Platform Roadmap

The Dev-Net reset and administrative foundation. I opened February by tearing down the development blockchain network, reseeding the genesis distribution, and rebuilding the administrator-facing dashboard against the freshly-initialised ledger. The reset surfaced a chaincode event-emission regression that had been silently dropping two event fields on transfer completion, which I fixed at the smart-contract layer and verified end-to-end by replaying a full registration and transfer flow. Alongside this I unified the API response contract across the remaining services so that every endpoint returned the same envelope shape, which removed a long-running source of frontend-backend friction.

Infrastructure continuity as a first-class concern. On February 5th I executed a 102 GB infrastructure backup of the production Kubernetes cluster, with cloud verification and a full integrity check across every Certificate Authority, every peer node, and the Hyperledger Fabric Orderer. The same session reclaimed roughly 242 GB of stale Docker layers and old build artefacts, and produced the planning document for the migration of the host operating system from AlmaLinux to Ubuntu 24.04 LTS. That plan eventually drove the mid-month infrastructure re-platforming.

The dual-codebase forensic audit. On February 8th I performed a comprehensive forensic comparison across the two parallel codebases we had accumulated. One was an older reference implementation, the other a newer architecture that had diverged across 300 plus files. The audit produced a sixty-issue catalogue spanning architectural drift, dead code paths, security gaps, and dependency inconsistencies. The outcome was a documented strategic decision to consolidate around the newer architecture and retire the reference implementation, a decision I carried into the following week.

Monorepo consolidation and framework migration. I consolidated what had been seven independent repositories into a single monorepo with Git submodules, migrated the frontend stack to Next.js 16 and Tailwind v4, and activated the CI/CD pipeline against the new layout. The framework migration was not cosmetic. Tailwind v4 changed the design-token expression model, which meant I had to pre-convert every colour and typography token before the build would pass. By the end of that stretch the continuous integration pipeline was running on every commit, and production deployment no longer required a human to run a shell script.

Backend engineering depth. In the same window I laid the domain-driven financial arithmetic foundation that the transfer pipeline would need later. It was a set of pure functions for working in the protocol's internal precision unit with strict BigInt semantics. I introduced a composite-event architecture to the CQRS layer so that related state changes could be projected atomically. I resolved a genesis-distribution root-cause bug that had been corrupting three participant accounts on every redeploy, stood up a dedicated user-profile microservice, and decomposed the single legacy CQRS worker into two focused workers, which simplified idempotency reasoning substantially.

Integration, UI quality, and liveness detection. The middle of the month was about pulling everything together. I led a multi-repository branch convergence across eight repositories, authored the wallet-dashboard architecture specification, and ran a four-track parallel overhaul of the participant-facing frontend covering currency formatting, shared components, form refinements, and a redesign of the liveness-detection experience. The liveness work was the most satisfying. I fixed a camera lifecycle bug where the video stream remained attached to an unmounted DOM node during a state transition, and I expanded the test suite to 110 passing tests. A full-stack audit at the end of the stretch identified 38 issues, of which I resolved 33 before the week closed.

Systemic bug remediation and the end-to-end test suite. On February 22nd I mapped and fixed 21 systemic bugs that had been silently propagating across three repositories. These fell into three classes: service-client misrouting where API routes dispatched requests to the wrong backend service, JWT-token leakage in error logging where seven route files dumped full authorisation tokens into server logs, and a chaincode-event field-name mismatch left over from an earlier schema

migration. In parallel I debugged the 48-test end-to-end suite from a wall of failures down to a complete 48-of-48 pass, working through distinct failure categories including responsive-layout ambiguity, Tailwind visibility semantics, and environment-variable bypass gating.

The same stretch delivered the place-of-birth feature across 15 files in the KYC pipeline, a four-layer persistence debug that traced a silent failure to a stale deployed container image, and a visual overhaul that brought the KYC surfaces into WCAG 2.1 AA compliance.

A dedicated file-storage microservice. KYC document uploads had been writing to ephemeral pod filesystem storage that vanished on every restart, silently losing user-uploaded identification. I designed and built a dedicated file-storage service from scratch as a clean-architecture system wrapping an S3-compatible object store, with SHA-256 deduplication, ClamAV virus scanning over INSTREAM, automatic image thumbnail generation, and presigned-URL downloads. Once the service was functional I put it through a production-readiness audit that surfaced 50 findings across four severity levels. The most critical was a timing attack on the API-key middleware, which I replaced with a constant time comparison. The next most critical was the absence of magic-byte validation on uploaded content, which I closed using the file-type library. I also rewrote the download pipeline to stream rather than buffer, reducing per-request memory from the file size to a single read chunk. I applied a sister 30-finding hardening pass to the marketing site's infrastructure the same day.

The exception handling architecture. A routine test revealed a silent fallback pattern. A missing environment variable had caused the identity verification service to fall back to local filesystem storage instead of the object store, and the failure had never surfaced. I treated this as a systemic philosophy problem rather than a local bug. I introduced six domain error classes each carrying the correct HTTP status code, rewrote the error handler middleware from a two category implementation to a five category system, wrapped every object storage and database-repository write with structured error propagation, and removed the silent default from the configuration schema entirely. After the rewrite the pipeline could no longer fail quietly.

A 22 module platform roadmap. On February 25th I reframed the question of execution priority as a system design exercise. I audited the entire codebase, which spanned 16 backend services, four frontend applications, and seven chaincode contracts with their Kubernetes manifests, and synthesised the findings into a 22 module roadmap with defined scopes, complexity estimates, and dependency relationships. I produced seventy plus architectural decisions covering everything from single sign-on to two-factor transfer security to a unified generic filter API.

Over the following two days I authored five enterprise specifications in parallel, totalling ~8,500 lines across participant administration, the post login hub, the wallet dashboard, the send flow, and the KYC review queue, then put all five through a cross referencing audit against the codebase and the decisions register. The audit produced ~60 findings, of which the most dangerous was a missing migration plan for existing refresh tokens in the single sign-on specification. I resolved all ~60 before the specs were executed.

First administrative modules delivered. I closed February by implementing the first two modules on the new roadmap. The individual participant administration module landed across seven work packages with a server-side data table that replaced client-side filtering and sorting, multi-select filter popovers, date-range presets, and a config-driven action panel. The KYC review queue module followed with SLA-tracking badges, reviewer-assignment workflows, document-viewer rotation with keyboard shortcuts, and face-verification score labels thresholded into pass, review, and fail tiers. I also propagated the standard table pattern across three management surfaces and brought each of them to WCAG 2.1 AA compliance.

March 2026: Design Systems, Core Modules, and the Blockchain Migration

Enterprise brand redesign and dark-mode remediation. I opened March by removing 49 MB of accumulated development artifacts from the repository, then ran a full enterprise brand redesign across 64 frontend files. A grep audit had identified roughly 727 hardcoded colour violations in the shared component layer that broke dark mode. I worked through them in two disciplined waves, reducing the violation count to nine intentional exceptions and eliminating the entire class of bugs. Alongside the redesign I introduced a global design-token architecture with a reusable component layer for inputs, cards, page layouts, and form stacks.

The send flow as a complete feature module. I then built the send flow end-to-end as a self contained feature module. The fiat conversion engine carried 174 currencies with their symbols, codes, and per-currency decimal rules. The conversion mathematics leveraged the protocol's gold genesis price model, which made the arithmetic deterministic and free of network dependency. I implemented the security verification step as a seven state finite-state machine covering every combination of PIN, two-factor authentication, and lockout. I added a recipient quick select component, rewrote the amount input as a bidirectional fiat-and-native dual entry, and integrated the whole module into a five step transfer flow.

Fixing the transfer pipeline end-to-end. Getting a transfer actually committed to the ledger required resolving a chain of problems. The initial submission failed because the outbox command was sending an internal database identifier to the blockchain instead of the blockchain-native identity string. I corrected the use case and introduced a dedicated domain error. The projector then crashed on completion because four fee related tables were missing from the database. I created those tables directly and built a transfer status endpoint end-to-end. A post transfer inspection then showed wallet balances inflated by a factor of one million, which I traced to four of the six CQRS projector handlers writing raw precision units without dividing through the conversion constant. I introduced a shared constant, fixed the four handlers, and ran a SQL correction across every affected wallet. I rounded out the week with a concurrent transfer architecture built on optimistic balance holds, cross tab synchronisation over the BroadcastChannel API, and a batch status endpoint, plus 42 new unit tests that brought the suite to 152 passing.

Specifications for treasury and institution onboarding. On March 5th I authored the specification for the government treasury module through 26 rounds of architectural Q&A. The first draft ran 1,314 lines. After a multi dimensional audit against the actual codebase exposed factual drift between the spec and the implementation, I produced a corrected second version at 1,535 lines. The module scope recalibrated from medium to large-plus once I had accurate endpoint counts. A parallel pass produced 31 new architectural decisions for the institutional onboarding module and split that specification into a core and an advanced document.

Single sign-on and the post-login surface. March's second week delivered the single sign-on architecture full-stack. I implemented OAuth2 with PKCE across the wallet, the administrative portal, and the institutional centre, with cross-application logout, server-side session tracking, and 15 minute access tokens backed by seven-day rotating refresh tokens. Twenty one tiered hardening fixes followed from a production audit, and every one of the six OAuth2 end-to-end scenarios passed cleanly after the fixes. In the same week I built the post-login hub surface as a multi-context router, rewrote the administrator-promotion UI against the new authentication model, fixed a subtle role-based access response bug, and refactored the component tree to satisfy the project's cyclomatic-complexity limits.

Live blockchain verification and the government module. The third week of March began with a multi-repository branch synchronisation and an enterprise Playwright test suite, then shifted to live-blockchain verification of the transfer flow with critical security fixes. The bulk of the work went into the government module, which I delivered across two days. Phases one through three landed on March 16th, and phases four through seven on March 18th. Phase four introduced inter government lending. Phase five covered transition and budget governance for electoral handovers. Phase six

added export and reconciliation tooling. Phase seven closed out security hardening, the CQRS worker pipeline, Kubernetes deployment, and smoke test verification.

Cross service token bridges and the eight repository synchronisation. Late March combined a cross-service SSO token bridge, four-module completion with live end-to-end bug discovery, an institutional-platform full-stack delivery, and a comprehensive single-sign-on security remediation across four codebases. I rewrote the orchestration architecture document from version two to version three, a substantive expansion of the role taxonomy, the rule set, and the discipline, and I ran a full synchronisation across all eight repositories that make up the project.

The Hyperledger Fabric to Hyperledger Besu migration. On March 25th I began the migration of the protocol off Hyperledger Fabric onto Hyperledger Besu. I planned the architecture, authored the Solidity implementation using the Diamond Standard (EIP-2535) for upgradeability, rewired the CQRS pipeline against the new chain's event semantics, and applied a platform wide security remediation pass. The diamond started at 9,260 lines at devnet go-live and grew to roughly 12,800 lines across 17 facets by the end of the reporting period. The verification suite grew through progressive milestones. First 434 tests, then 526, then 683, then a full 758 passing before deployment. On March 26th I ran the devnet go-live marathon. The steps were Diamond deployment, a pre-deployment security audit that covered the contracts and the orchestration, a separate production readiness audit that covered the operational envelope, CQRS pipeline validation against live events, and a final remediation pass that closed every outstanding finding.

April 2026: Production Readiness, the Marketing Track, and the Design Marathon

End-to-end test resurrection. April opened with a resurrection of the Playwright MCP (Model Context Protocol) end-to-end test suite that had degraded during the Besu migration. Getting it back to green took 55 iterations of targeted debugging and surfaced four real production bugs along the way. The process was slow and repetitive, but the result was a tripwire I could trust on every subsequent change.

Production readiness sprint. I then ran a production readiness sprint focused on the systemic weaknesses the migration had exposed. I completed the genesis distribution pipeline, eliminated 430 unsafe type assertions and 235 unsafe catch blocks across 258 files, and locked the codebase under TypeScript's strict compliance mode. I verified the three-eyes government approval flow end-to-end, resolved a class of systemic migration bugs where the new chain's event format had not been fully absorbed by the projectors, and closed two modules that had been sitting at ninety percent complete.

The automated marketing daemon. The first full week of April was given to the autonomous marketing pipeline. I began with an infrastructure constraint analysis, pivoted the architecture from a scheduled job model to an AI-agent model, and designed a daemon based pipeline running on Ubuntu 24.04 that composes social media posts, selects imagery, and publishes on a defined cadence. Alongside the daemon I ran a full-stack live validation pass that executed 398 end-to-end tests. Of those, 324 passed on the first run, and I worked through the remaining failures across the next two days. I rebuilt the Docker image pipeline, stabilised three misbehaving services, fixed a content security policy hydration issue that had been breaking the public site on mobile, and tuned the autonomous agent to reduce its cost per run.

Administrative modules and the CI/CD redesign. The second full week covered a production hardening sprint on the CQRS pipeline, a dev-net genesis reset with live user operations verification, and a participant administration module sprint that delivered 30 protected endpoints, a Docker based node agent, 79 passing tests, and the remediation of a 52 finding production audit. I also ran a credibility remediation on the marketing website, formalised the design system, swept 25 plus pages

into the new design language, and rebuilt the typography system with fluid clamp based scales. The single largest operational deliverable of the week was the CI/CD pipeline redesign. The project had been deployed through a fragile Apache based mechanism that had started failing silently, and I rebuilt it as a GitHub Container Registry blue-green deployment pipeline. Getting it to green required 18 pipeline runs. Each failure taught me something specific about Docker layer caching, the Prisma client generation step, or environment variable scoping inside GitHub Actions.

The design marathon on April 19th and 20th. The final two days of the reporting period ran three streams in parallel at unusual density. I opened the 19th with a defect fix on the public pre-registration flow, where the success panel had been declaring users on the waitlist before their confirmation email had been clicked. The feature looked healthy from the outside and was silently dropping users. I rewrote the success state into a "check your email" panel with a three-step checklist and a 72-hour expiry notice, wired the submission into a per-form registry, replaced the generic email template with a waitlist-voiced version, and closed six further items from a self-run accessibility audit. I then ran a wireframe design marathon authoring the participant-wallet surface across 65 screens, the institute-wallet surface across 37 screens, and the government-wallet surface across 41 screens, with a catalogue of 26 reusable primitives. On the 20th I built the institutional unified-admin surface at the desktop-primary viewport, shipping a 2,301 line wireframe file across 34 screens with five new primitives. A polish pass on that surface caught a type-guard violation where an institution-scoped primitive had leaked into a government-scoped screen, and (more interestingly) caught a narrative contradiction where one screen credited an approval to one administrator while a later screen credited the same approval to a different one. The tooling surfaced candidates. I was the one who read the wireframe as a continuous story and adjudicated which administrator should own the approval.

Parallel tracks: the administrative console and the marketing-website rebuild. In parallel with the wireframes I authored the specification for the internal administrative console over 600 lines in 15 sections, and landed four phases of scaffolding: the data plane, the dual-write layer, the authentication primitives and routes, and the shell with the first authentication pages. Two silent CI failures surfaced during the scaffolding: a Redis bootstrap error throwing at module load time, and a Prisma client missing from the Docker builder stage because the schema was not being copied into the dependencies layer. I fixed both. The third track was the marketing website content parity rebuild, which I executed as 54 commits across five disciplined phases: a Playwright and axe-core harness, primitive extraction, mechanical hygiene, the Global Impact section, and the Gen Z section, with a multi-stage review gate in front of each phase. The content shipped. Then the founder looked at the live page and pointed out that the new sections did not look like the rest of the site. I redesigned six components with the content held constant, wrapping them in eyebrow pills, gradient display headings, staggered reveal animations, and the site's alternating section backgrounds. The clearest new lesson I took from this period is that content parity is not the same as brand parity, and that a specification must encode both. I closed the session with a research pass on the administrative command-centre surface, mapping roughly 60 existing routes and 12 concrete gaps that seeded the 54 screen command-centre wireframe set I shipped the next day, alongside a future modules roadmap and a cross-cutting primitives catalogue that together closed out the seven phase wireframe corpus.

The marketing and business operations track. Alongside the engineering and the design work, I ran the brand, marketing, and business-operations side of the venture in parallel with the founder almost every day. That included brand guideline authoring, landing-page copywriting and messaging strategy, pre-registration waitlist activation, data visualisation for the public site's Gen Z and Global Impact sections, a Foundation-naming ruling that I then pinned into every downstream artifact, social-media content strategy, stakeholder narrative work, and commercial coordination. The autonomous marketing daemon I built in April is the productisation of that parallel track: a mechanism that turns a weekly marketing rhythm into a repeatable operational pipeline.

2. Description of Internal/External Training Undertaken

The training during this period was almost entirely project-driven. Every new technology arrived because a specific problem needed it, which meant I learned it under real consequences rather than tutorial conditions.

Enterprise specification authoring and multi-dimensional spec auditing. The 22 module roadmap and the 8,456 line specification suite forced me to treat specification writing as a first-class engineering discipline. I learned to encode acceptance criteria rigorously, separate architectural decisions from implementation sketches into a dedicated register, and put each specification through an audit pass against the actual codebase before committing it to execution. Catching 59 findings across five specifications before any code was written taught me that a well-authored specification is cheaper to fix than the code it produces.

An agentic, AI-augmented engineering workflow. The single largest shift in how I work this year is that I operate inside a disciplined agentic workflow that I designed and am still refining. The substrate is Claude Code and the broader family of open-source agentic coding tools emerging alongside it, but the workflow itself is my own system, captured in a living orchestration architecture document that I have rewritten three times across the placement, most recently to a version-three that expanded the reviewer-role taxonomy and tightened the rule set. What I built is not “ask an AI for code.” It is a multi-stage review discipline with specific reviewer roles, explicit file-boundary contracts for parallel work, canonical-source pins so agents cannot drift from the ground-truth specifications, a plan-and-approve gate before any implementation runs, and a mandatory validation pass after implementation. I author every brief myself, adjudicate every finding on its merits, accept or rescope each remediation, and own the final sign-off. The agents surface candidates. I am the one who decides.

The mechanics underneath the discipline. Claude Code gives me the primitives to operate that discipline at real speed: parallel subagent dispatch with zero-conflict file boundaries, hooks for deterministic automation, a memory system that carries context across conversations, slash commands for recurring patterns, and per-project CLAUDE.md files that encode the house style and constraints. I treat these the way a serious engineer treats Git, Docker, or continuous integration. They are part of the professional stack. My own “Super Engineer Review Board” pattern is a repeatable choreography that runs on top of those primitives: a coverage-and-accuracy reviewer, a template-and-precedent reviewer, and a narrative-and-reception reviewer, each with its own brief and its own file-read mandate, all dispatched in parallel, with synthesis and adjudication retained by me. Variations of the same pattern sit behind every specification audit, every security audit, every pre-deployment readiness check I ran this period.

How this multiplied the work. Concrete examples. I ran the 21-bug systemic remediation on February 22nd across three repositories as a three-tier parallel choreography (brief-writing, implementation, validation) across zero-conflict file boundaries, and my validation tier caught a JSX structural violation the implementation tier had missed. I drafted the five enterprise specifications on February 25th (8,456 lines across five modules) in parallel under the same discipline, then put them through a cross-referencing audit against the codebase and the decisions register that surfaced the 59 findings I resolved. I built the blockchain migration’s 758-test verification suite under the same choreography. I gated the 280-screen wireframe marathon on April 19th and 20th with a combined canonical-brand, UX-completeness, and Playwright-rendered polish board at every phase, and that pattern is what surfaced the narrative contradiction I adjudicated on the 20th. None of these would have been possible at the velocity they ran without the agentic substrate. All of them required the discipline I built on top of it to avoid silent drift, incorrect defaults, or plausible-but-wrong output reaching production.

What I think this means for software engineering. I am aware that I am working inside a genuine inflection in how software gets built. A disciplined operator with a well-designed agentic workflow can now run at a scale that used to require a team. That is not a slogan. It is the operational reality I have been producing inside every day this period: a protocol migration, several modules, an administrative console, a marketing daemon, a website rebuild, and a design system authored in parallel by one person who is also running the marketing and business operations of the same venture. The counterweight is that the multiplier applies equally to carelessness. A brief I did not tighten, a validation tier I skipped, a canonical source I did not pin: any of those would have produced confident-looking, plausible-sounding, silently-wrong output at the same multiplier. The professional skill I am genuinely developing this period is not “how to use Claude Code.” It is how to operate a team-of-agents responsibly: framing briefs that produce verifiable output, designing review stages that catch what the previous stage missed, and treating my own judgement as the terminal authority that no tool can substitute for. This is how I expect startup-scale software engineering to be done for the rest of my career.

Solidity and the Diamond Standard. Migrating off Hyperledger Fabric onto Hyperledger Besu was a crash course in EVM-native development. I had to internalise Solidity’s storage layout, gas considerations, and the proxy and facet patterns of the Diamond Standard (EIP-2535) well enough to implement the entire protocol as a single diamond, starting at 9,260 lines of contract code. The migration also taught me how different event semantics are between a Fabric private-chain model and an EVM public-chain model, and how the CQRS projector pipeline has to be rewired to match.

CQRS pipeline engineering. The composite-event architecture and the outbox pattern matured into tools I reach for by default. I learned to think of each projector handler as an idempotent function over an event, to push related state changes into a composite event so they project atomically, and to treat unit conversion at handler boundaries as a first-class concern rather than an implementation detail.

Clean Architecture at scale. Every new service this period was built on the four-layer clean-architecture model (domain, application, infrastructure, interface) and the ports-and-adapters pattern. The discipline paid off. When I had to swap the storage adapter from the filesystem to the object store, or swap an authentication strategy from a per-service implementation to the unified single-sign-on, the change was localised to the infrastructure layer.

Object-storage engineering. The dedicated file-storage microservice taught me the S3 API at real depth: multipart uploads, presigned URL generation, streaming reads, human-readable key design for administrator observability, and cross-bucket isolation. The timing-attack mitigation on the API key, the magic-byte validation, and the ClamAV INSTREAM integration each landed specific security lessons that I now carry into every new service.

Enterprise testing discipline. I extended the testing trophy into the new services with Vitest and Testcontainers, added Playwright end-to-end coverage across the wallet and the administrative portal, and treated the test suites as the first citizens of production readiness rather than afterthoughts. The 48-of-48 pass, the 152-test transfer suite, and the 758-test Besu migration suite all became credibility anchors rather than just coverage numbers.

Design systems and wireframing as a first-class discipline. This was new for me. I went from using a designer’s output to authoring the design system myself: tokenised colour and typography, desktop-versus-mobile shell discipline, a catalogue of reusable primitives with enforced type guards so that government primitives cannot render inside institutional screens, and the lesson that a design specification has to encode both content and brand. The 280-screen wireframe set at the end of the period was the clearest artefact of this training.

Brand, marketing, and business operations. Marketing-content development is 20% of my official role per my offer letter, but the scope and depth of that work expanded significantly this period. I authored brand guidelines, wrote landing-page copy and waitlist email voices, produced data visualisations for a public site, participated in founder-level decisions about Foundation naming and

messaging strategy, and instrumented the public pre-registration funnel. Treating this as a training domain rather than a side activity was one of the most useful mental shifts of the period.

Security engineering. Specific lessons landed. Constant-time comparison to close timing attacks, magic-byte validation on uploads, streaming rather than buffering to avoid memory-exhaustion attacks, OAuth2 PKCE for public clients, content-security-policy hydration discipline on the public site, and the general principle that a system that fails silently is less safe than one that fails loudly.

CI/CD maturity. The Apache-to-GitHub-Container-Registry pipeline redesign taught me blue-green deployment, Docker multi-stage build layering with Prisma generation inside the builder stage, and environment-variable scoping inside GitHub Actions. The 18 pipeline runs required to reach green were the training.

Blockchain-migration engineering. Identity mapping between internal identifiers and blockchain-native addresses, unit conversion as a migration concern, and audit-trail continuity across a chain swap are all things I now understand as specific disciplines rather than general concerns.

Technical documentation and narrative-driven work records. My daily work records continued to evolve as a practice. Writing them as narratives rather than commit lists is both the input to reports like this one and a form of thinking out loud that has repeatedly caught errors before they reached production.

3. Reflection on Your Learning (e.g., skills and knowledge gained)

These three months changed how I think about what finished software actually means. The word that kept coming back to me was **silent**. A system can pass every visible check and still be quietly wrong, and it is my job as the engineer to make the silent things loud before they reach a user.

The pre-registration confirmation funnel on April 19th is the clearest example. The form submitted cleanly, the success panel declared victory in green, and the user went about their day believing they were on the waitlist. They were not. The Google Sheets row was only written once they clicked the link in the confirmation email, and if the email landed in spam they were never added. Nothing in the stack logged an error. Nothing in the tests failed. The only thing that surfaced was the decision to trace the flow end-to-end. The fix was not hard. The lesson was that I now check every “success” state for whether it is actually telling the truth.

The unit-standardisation bug in March was the more expensive version of the same lesson. Four of the six CQRS projector handlers were writing raw precision units into the cached wallet balance column without dividing by the conversion constant. Balances inflated by a factor of one million, and the system never complained because no constraint was being violated. The numbers were numbers, they just meant different things in different handlers. I corrected the handlers, introduced a shared constant to prevent regression, and ran a SQL correction across every affected wallet. The lesson I took from it is that unit conversion at handler boundaries is a load-bearing concern, and sharing a constant across handlers is cheap insurance against a class of silent corruption that would otherwise be caught only by an angry user.

The brand-redesign pass on April 20th taught me something I did not know about specifications. I had written careful content-parity specifications for the public website that named every paragraph, every data source, and every verbatim disclaimer. The content shipped. Then the founder looked at the live page and said, in his exact words, “the flaw there is it doesn’t match our site branding, our design patterns, our typography or the rest of the pages.” He was right. I redesigned six components with the content held constant, and I carry forward the lesson that a specification must encode both the content and the brand, because if it only encodes one the other is not checked.

The design review discipline proved its worth in a specific way on the 20th. The wireframe polish pass caught a narrative contradiction where one screen credited an approval to one administrator and a later screen credited the same approval to a different one. Neither screen was wrong in isolation. The contradiction only surfaced when a reader followed the same transfer across the file. That catch was only possible because I was reading the wireframe as a continuous story, and the reason I was doing that was because I had deliberately set up the review pass to do exactly that. Discipline beats instinct, and designing the discipline is itself engineering.

The last lesson is about running three streams in parallel. Engineering, design, and marketing all moved during this period, and the days that went poorly were the days I tried to do all three without pre-allocating attention. The days that went well were the ones where I committed the morning to one stream and accepted that the other two would have to wait. I am still learning how to manage that tension, but I now know it is a real skill rather than a failure of focus.

The deeper lesson behind all of this is about what a disciplined agentic workflow changes about software engineering itself. I spent most of this period operating inside a system I designed on top of Claude Code and the wider family of agentic coding tools, and the effect on what one engineer can ship is not subtle. I moved a protocol migration, several modules, an administrative console, a marketing daemon, a full website rebuild, and a 280-screen wireframe set forward across roughly eleven weeks while I was also running the marketing and business operations of the same venture. That scale is not a result of working harder. It is a result of a new class of professional discipline: framing briefs that produce verifiable output, gating parallel work with multi-stage review, pinning canonical sources so agents cannot drift, and treating my own judgement as the terminal authority that signs the work off. I am aware that most of the conversations happening in the industry right now about agentic coding are either euphoric or dismissive, and I have become convinced that the honest answer is in the middle. The tooling multiplies both care and carelessness with equal precision. The engineer's job is to be the one that supplies the care. That is the skill I was actually training for this period, and it is the skill I expect to define startup-scale software engineering for the rest of my career.

4. Identify Any Areas Requiring Improvement

My previous report named five gaps: chaos engineering and resilience testing, performance testing and optimisation, GitOps and true infrastructure-as-code, security auditing and penetration testing, and cost optimisation and resource management. It is worth being honest about which ones moved.

Chaos engineering. Still open. The stack is now significantly more resilient on paper than it was in January. The new blockchain layer has its own consensus mechanism, the services are more cleanly separated, and the error-handling architecture means failures are loud. But I have not yet deliberately broken anything in controlled conditions. This needs to happen in the next reporting period.

Performance testing. Still open, and more pressing now. The end-to-end transfer pipeline is functional and the design system is stable. The next honest question is how it behaves under real load. I have not yet set up a load-testing harness.

GitOps. Partially addressed. The CI/CD pipeline redesign and the GitHub Container Registry blue-green deployment are significant steps toward it, but the Kubernetes cluster is still not syncing declaratively against Git. That remains the destination.

Security auditing. Still open. I have continued to close findings from my own multi-dimensional audits: 50 in the storage service, 30 on the marketing site, 59 across the specifications, 52 on the administrative module. But I have not yet engaged an external auditor. The protocol is financially material enough that this gap is no longer acceptable.

Cost optimization. Unchanged. The stack still runs on predominantly free tiers.

Two new gaps surfaced this period, and a third that I only noticed by living it.

Observability for the autonomous marketing daemon. I built the daemon and deployed it, but I cannot yet see its failure modes, its cost per run, or the quality of the signal it produces. It is exactly the kind of system that fails silently, exactly the class of problems this period taught me to refuse.

Time management across three parallel streams. This is the most subtle gap. I now have several weeks of data on which days dropped context, and the pattern is that context-dropping correlates with trying to hold all three streams open simultaneously rather than pre-allocating attention. I need a structured mechanism for this, not just instinct.

5. Your Goals or Targets for the Next Couple of Months

Every goal below is tied either to a Section 4 gap or to work that is already in flight and needs to land.

Ship the unified decision-panel primitive. Across KYC review, institutional verification, partner onboarding, and treasury activation, the approve-and-reject dialogs are currently duplicated hooks and components. Consolidating them into a single primitive closes a design-specification gap and reduces the surface area of anything that could go silently wrong.

Build the command-centre and compliance-and-marketplace modules against their shipped wireframes. Phase five and phase six wireframes landed on April 20th. The next step is turning those wireframes into production modules, with the cross-cutting primitives catalogue as the shared foundation. Closing both is the test of whether the design discipline I built this period holds up under a real build.

Productionise the autonomous marketing daemon. Instrumentation, cost controls, failure-mode alerts. This directly closes the observability gap I named above.

Bring the new blockchain stack to a public testnet with external users. This forces real-world failure modes into the open and is the most valuable version of the chaos-engineering gap I have been carrying since January.

Commission the first external security audit. The protocol is now mature enough and the financial exposure material enough that my own audit discipline is no longer sufficient. This closes the security-audit gap.

Run the first end-to-end chaos drill in the devnet environment. Kill database replicas, sever network segments, deliberately revoke credentials, and document what recovers on its own versus what requires manual intervention. This is the chaos-engineering gap in its direct form.

Close the remaining Wave B items on the marketing website. HSTS and strict content-security-policy, accessibility essentials, SEO architecture, and GDPR consent. Treating these as a spec-driven wave rather than a drive-by pass is how I will address the new brand-and-design specification gap.

Formalise contract-test coverage across services. Pact or an equivalent, on every cross-service boundary. This continues the testing-trophy discipline from the prior period and hardens the seams.

Stand up a load-testing harness. This closes the performance-testing gap. A k6 or equivalent harness exercising the participant-wallet and transfer flows under realistic concurrency, with headline

thresholds on throughput and p95 latency that have to hold before any external-user onboarding.

Move the Kubernetes cluster onto GitOps. This closes the GitOps gap I have been carrying since the previous report. Either ArgoCD or FluxCD syncing from the monorepo so merges, not humans, drive deployment.

Adopt a structured mechanism for three-stream time management. This closes the time-management gap I named in Section 4. A written daily attention budget committed to before I start work, not reconstructed in reaction to it.

Onboard the first cohort of real pre-registration waitlist members. This is the integration test for everything above. It is the only test that matters.

6. Describe and Evaluate Your Own Skills in Relation to the Graduate Attributes and the Professional Skills Needed in the Sector

These months gave me evidence of growth across the attributes I identified in the previous report, and clarity on which are now genuinely my own rather than borrowed from tutorials.

Problem-Solving and Systematic Debugging. The unit-standardisation bug, the silent-fallback discovery in the storage pipeline, the 55 iterations to resurrect the end-to-end test suite, and the adjudication of the wireframe narrative contradiction are all specific cases where the instinct was not “try random things” but “form a hypothesis, test it, eliminate possibilities.” The four-layer persistence debug on the KYC place-of-birth field in February is the clearest single example. It took four distinct layers of investigation to reach the actual cause (a stale deployed container image), and the work at each layer was structured rather than improvised.

Infrastructure and Operations Thinking. The blockchain migration, the CI/CD pipeline redesign, the Docker multi-stage rebuild, and the autonomous daemon infrastructure all exercised operational thinking in a way earlier periods did not. I am now comfortable asking the operational questions (“what happens when this fails at 2 a.m.?”) as a first-pass filter on design decisions rather than as a late-stage audit.

Security-First Design. Constant-time API-key comparison, magic-byte validation, streaming downloads, OAuth2 PKCE, content-security-policy hydration, zero-trust service boundaries, and the enterprise exception-handling architecture that refuses silent fallbacks. I am not a security specialist, but the reflex of asking “how does this fail, and who benefits from its failure?” is now automatic.

Technical Communication and Documentation. The 22-module roadmap with 69 architectural decisions, the 8,456-line specification suite, the architecture rewrite to version three, the 280-screen wireframe set, the daily work records that read as narratives, and the two prior placement reports plus this one are the visible evidence. What I learned this period is that good technical communication is not just writing a document. It is building an artefact that someone else can review, audit, and learn from, and that survives the moment it was written.

Project Ownership and Execution. The Fabric-to-Besu migration was a decision I drove end-to-end, including the honest pivot to change blockchain platforms when the evidence said the existing one could not carry the protocol’s next phase. The 22-module roadmap was my synthesis, executed against my velocity. The phase-gated delivery of every major module (send flow, government treasury, institutional-onboarding, administrative console) was me owning the delivery from specification through implementation through audit through deployment.

Adaptability and Continuous Learning. Picked up cold in this period: Solidity and the Diamond Standard, Hyperledger Besu, Tailwind v4, Next.js 16, wireframing and design-system authoring as a discipline, brand and marketing operations. The rate at which I can read a new technology's documentation, build something meaningful with it, and then critique my own implementation has noticeably improved since January. The most consequential adaptation was learning to operate a disciplined agentic engineering workflow on top of Claude Code and the emerging open-source agentic tooling, and to treat that orchestration practice as a professional stack I am responsible for, not as a convenience feature.

Architecture Design and System Evolution. The Solidity Diamond, the clean-architecture discipline carried across every new service, the composite-event CQRS pattern, the design-system consolidation across 64 frontend files, the orchestration architecture rewrite. The evidence is that the architectural decisions I make now tend to hold up to audit rather than collapse under it.

Collaboration and Stakeholder Management. This is where the marketing and business track lives most load-bearingly. Working directly with the founder across engineering, design, and commercial decisions required translating between three different kinds of risk (technical, brand, commercial) on the same call. Authoring the Foundational rulings, running the pre-registration funnel, shaping the social-media content strategy, and producing the autonomous marketing daemon as a commercial operational system rather than a technical experiment all exercised this attribute. I am also now consciously producing artefacts (specifications, plans, daily work records) that are reviewable by others, which is its own form of collaboration.

Professional Ethics and Responsibility. A protocol that handles real value demands ethical discipline. The refusal to ship silent failures, the careful design of the tiered registration flow so that participants cannot be accidentally exposed to financial operations they have not yet consented to, and the honest acknowledgement of mistakes in my own work records all reflect this. I am aware that I am building something whose failures would affect real users, and I design against that possibility from the first paragraph of a specification.

Quality and Professionalism. The review discipline that caught the narrative contradiction, the audit passes that closed 50-plus findings before deployment, the refusal to ship the end-to-end suite at anything less than 48-of-48, the 18 pipeline runs to get the CI/CD redesign to green, and the brand-redesign pass that treated "content shipped" as insufficient when "content shipped and looks like the site" was the actual requirement. Quality for me is no longer about polish. It is about refusing to accept the version of "done" that isn't actually done.

Looking across these attributes, the single biggest shift since January is that I stopped being an engineer who had to learn new domains by stepping outside the work, and became an engineer whose work is the learning. Design, marketing, brand, infrastructure, security, and operations have all become parts of the same craft rather than separate departments. The next period will be about operating that integrated craft under real users and real load.

Performance for the month:

Once you have addressed the points (outlined in questions 1-5), ask your supervisor to assess your performance for the month using the following table.

	<i>Below expectation</i>	<i>Satisfactory</i>	<i>Above expectation</i>	<i>Outstanding</i>	<i>N/A</i>
Attitude to work			✓		
Quality of work				✓	
Productivity				✓	
Literacy and Communication skills			✓		
Critical Thinking skills			✓		
Reliability		✓			
Global Outlook			✓		
Suitability of future goals (refer to question 5)			✓		
Overall Achievement			✓		

Workplace Supervisor Feedback / Review:

Manazir's performance during this reporting period has been exceptional, resulting in an 'Above expectation' overall achievement. His Productivity and Quality of work are Outstanding, demonstrated by the delivery of complex, multi-stream projects including the full Hyperledger Fabric-to-Besu migration, a dual codebase consolidation, and a 280-screen wireframe set. He proactively identified and fixed systemic weaknesses, such as the silent pre-registration failure and unit-standardization bugs, showing a strong Critical Thinking ability and a refusal to accept quietly wrong systems.

His **Attitude to work** is exemplary; he operates with a professional discipline, framing a sophisticated 'agentic' workflow that significantly multiplied his output and allowed him to manage engineering, design, and marketing tracks concurrently. His **Literacy and Communication skills** are strong, evidenced by the high-quality, narrative-driven specifications and reports.

The only area rated 'Satisfactory' is **Reliability**, which reflects the acknowledged challenge of balancing three demanding parallel streams. Manazir has correctly identified 'structured time management' as a necessary goal for the next period to ensure sustained reliability across all functions. His **Future Goals** are well-suited to closing remaining gaps (e.g., chaos engineering, external security audit) and moving the protocol to a public testnet, fully deserving of the 'Above expectation' rating.

Workplace Supervisor Signature:



M Azim Abdul Majeed

Date: April 23rd, 2026

Student Name & Signature:



Manazir Ali

Date: April 21st, 2026